

# Modelling of a VLM based Nonlinear Control of a mobile robot for precise navigation and action in inspection operations

Github link: <https://github.com/AkhilaRao0002/VLM-Based-Nonlinear-Control-for-Precise-Mobile-Robot-Navigation-for-inspection-operations>

## Objective

The objective of this project is to develop an intelligent vision-based navigation and inspection system for a mobile robot that can perceive a target object and surrounding obstacles from RGB/RGB-D visual data, estimate their 3-D spatial information, and autonomously navigate to a safe inspection position using Nonlinear Model Predictive Control (NMPC).

The system integrates Vision-Language Model (VLM)-based scene understanding, RGB-D geometric reconstruction, obstacle-aware path planning, and NMPC-based motion control into a unified navigation pipeline.

## Specific Objectives

1. **Scene Understanding**  
Identify the target object and relevant obstacles from the input scene using VLM-based perception or the provided synthetic ground-truth perception interface.
2. **3-D Spatial Estimation**  
Use RGB-D data and camera intrinsic parameters to convert image-space bounding boxes and depth information into **metric 3-D coordinates**.
3. **Robot Coordinate Transformation**  
Transform the estimated target and obstacle geometry from the camera coordinate system into the robot's coordinate frame.
4. **Inspection Pose Generation**  
Determine a suitable inspection position at a specified distance from the target and calculate the required robot heading so that the robot faces the target.
5. **Obstacle-Aware Navigation**  
Construct a metric obstacle map and generate safe intermediate waypoints when the direct path to the target is blocked.
6. **NMPC-Based Motion Control**  
Employ Nonlinear Model Predictive Control to continuously determine the robot's linear and angular velocities while considering:
  - Target position error
  - Heading error
  - Obstacle clearance
  - Control effort
  - Terminal positioning accuracy
  - Progress toward the goal

## 7. Collision Avoidance and Safety

Maintain a predefined safety distance from obstacles and ensure collision-free navigation throughout the simulation.

## 8. Performance Evaluation

Evaluate the navigation system using quantitative metrics such as:

- Inspection goal achievement
- Final position error
- Final heading error
- Path length
- Minimum obstacle clearance
- Collision count
- Linear and angular velocity
- NMPC computation time

The overall objective is therefore to demonstrate a complete perception-to-action robotic navigation pipeline in which visual scene understanding is converted into metric spatial information and subsequently used by an NMPC controller to achieve safe, precise, and autonomous target inspection.

# Novelty

The novelty of this project lies in the integration of vision-language perception, RGB-D geometric reasoning, safe waypoint generation, and nonlinear model predictive control into a single target-inspection navigation framework.

Unlike a conventional navigation system that assumes a predefined map or manually specified target coordinates, the proposed approach establishes a pipeline from visual perception to autonomous robot motion.

## Key Novel Contributions

1. **VLM-Driven Scene Understanding:** The system uses a Vision-Language Model to interpret the scene based on a natural-language inspection task. This allows the robot to identify the target object, relevant obstacles, and navigation-related information rather than relying solely on conventional object detection.
2. **Integration of VLM Perception with RGB-D Geometry:** The detected 2-D bounding boxes are combined with aligned depth information to estimate the target and obstacle locations in metric 3-D space. This bridges the gap between semantic scene understanding and physical robot navigation.
3. **Automatic Inspection Pose Generation:** Instead of navigating directly to the detected object's center, the system calculates a safe inspection pose at a preferred distance from the target and automatically determines the robot heading required to face the target.
4. **Semantic-to-Metric Navigation Map:** Semantic information such as *"table"* and *"chair"* is converted into a geometric representation suitable for motion planning. Thus, the system connects high-level semantic perception with low-level geometric control.

5. **Obstacle-Aware Intermediate Waypoint Planning:** When the direct path to the inspection pose is blocked, the system generates safe intermediate waypoints around the obstacle. This enables the NMPC controller to handle a navigation task that cannot be solved safely by simply driving toward the final goal.
6. **NMPC-Based Closed-Loop Control:** The robot does not simply follow a predefined trajectory. NMPC repeatedly predicts future robot motion and optimizes the control inputs while considering position, heading, obstacle clearance, control effort, and progress toward the goal.
7. **Safety Integrated Directly into the Controller:** Obstacle clearance is incorporated into the NMPC cost function through an influence region and a stronger safety-distance penalty. This encourages the robot to begin avoiding obstacles before reaching the critical safety boundary.
8. **Quantitative End-to-End Evaluation:** The complete system is evaluated using both navigation-performance and safety metrics, including collision count, minimum obstacle clearance, path length, final position error, final heading error, velocity limits, and NMPC computation time.

## Methodology

The proposed system follows an end-to-end Perception → 3-D Geometry → Inspection Pose → Safe Planning → NMPC Control → Evaluation architecture. The methodology combines semantic scene understanding with metric RGB-D information and closed-loop nonlinear control.

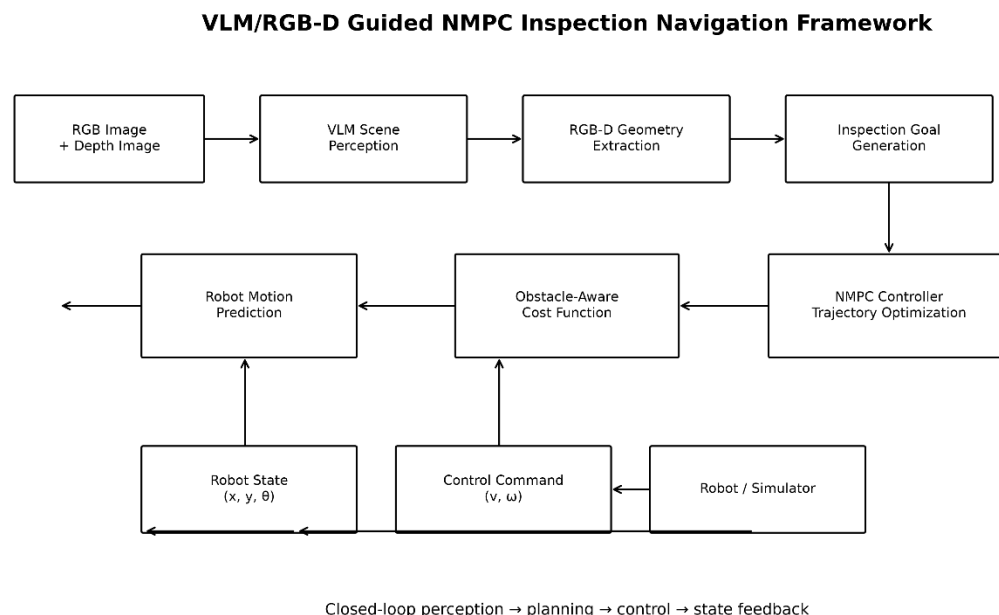


Figure 1: Overall System Architecture

### Input Scene and Perception

The system takes two primary inputs:

- RGB image
- Aligned depth image

The RGB image provides semantic information about the environment, while the depth image provides metric distance information.

The perception module identifies:

- Target object
- Target bounding box
- Relevant obstacles
- Obstacle bounding boxes
- Whether the direct path is blocked
- Navigation-related scene information

For example, in the experimental scene, the perception system identifies:

Target:

chair

Bounding box = [85, 145, 155, 270]

Obstacle:

table

Bounding box = [70, 150, 236, 376]

The perception output therefore provides the semantic information required by the downstream navigation system.

### **RGB-D Processing**

After obtaining the target and obstacle bounding boxes, the system uses the depth image to recover their spatial information.

The RGB-D processor receives the camera intrinsic parameters:

$f_x, f_y, c_x, c_y$

where:

- $f_x$  = focal length in x direction
- $f_y$  = focal length in y direction
- $c_x$  = principal point x coordinate

- $c_y$  = principal point y coordinate

The depth value is converted into meters using the depth scale:

$$Z = d \times s$$

where  $d$  is the raw depth value and  $s$  is the depth scale.

### Robust Depth Estimation

Instead of relying on a single depth pixel, the system extracts a small neighborhood around the selected pixel.

The valid depth values are filtered according to:

$$Z > 0$$

and finite values are retained.

The median depth is then calculated:

$$Z_{\text{robust}} = \text{median}(Z_1, Z_2, \dots, Z_n)$$

This reduces the effect of invalid or noisy depth measurements.

### Pixel-to-3-D Conversion

For a pixel and depth, the corresponding 3-D point is calculated using the pinhole camera model:

$$X = (u - c_x)Z / f_x$$

$$Y = (v - c_y)Z / f_y$$

$$Z = Z$$

Therefore,

$$P_{\text{camera}} = [X \ Y \ Z]$$

For the experimental target, the resulting target center is:

$$P_{\text{target}} = [-4.00, -0.66, 5.00] \text{m}$$

in the camera coordinate system.

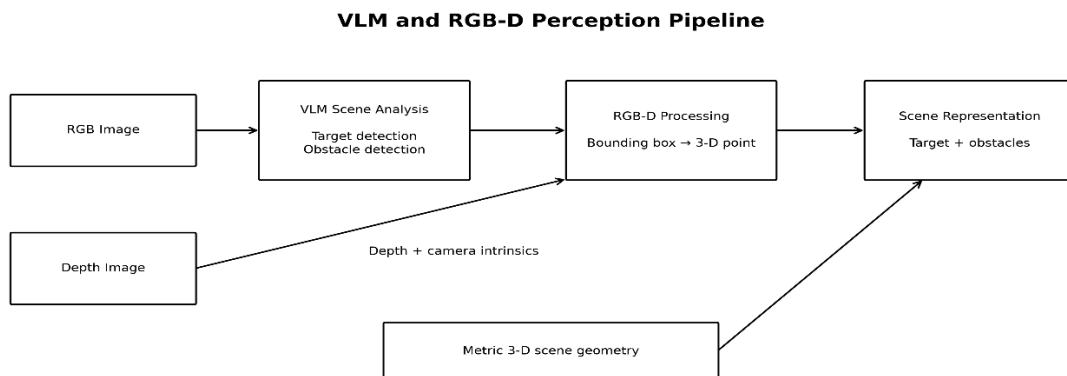


Figure 2 : RGB-D processing pipeline showing bounding-box detection, robust depth extraction, and pixel-to-3-D coordinate conversion.

### Camera-to-Robot Coordinate Transformation

The camera coordinate system and robot coordinate system use different axis conventions.

The system assumes:

#### Camera frame

- X : right
- Y : forward

#### Robot frame

- x : forward
- y : left

Therefore, the transformation is:

$$x = Z$$

$$y = -X$$

or

$$[x \ y] = [Z \ -X]$$

For the detected chair:

$$X = -4.0 \quad Z = 5.0$$

Giving:

$$x = 5.0$$

$$y = 4.0$$

Thus the target is approximately:

$$P_{\text{robot}} = [5.0, 4.0] \text{m}$$

from the robot's starting position.

## Camera-to-Robot Coordinate Transformation and Inspection Pose

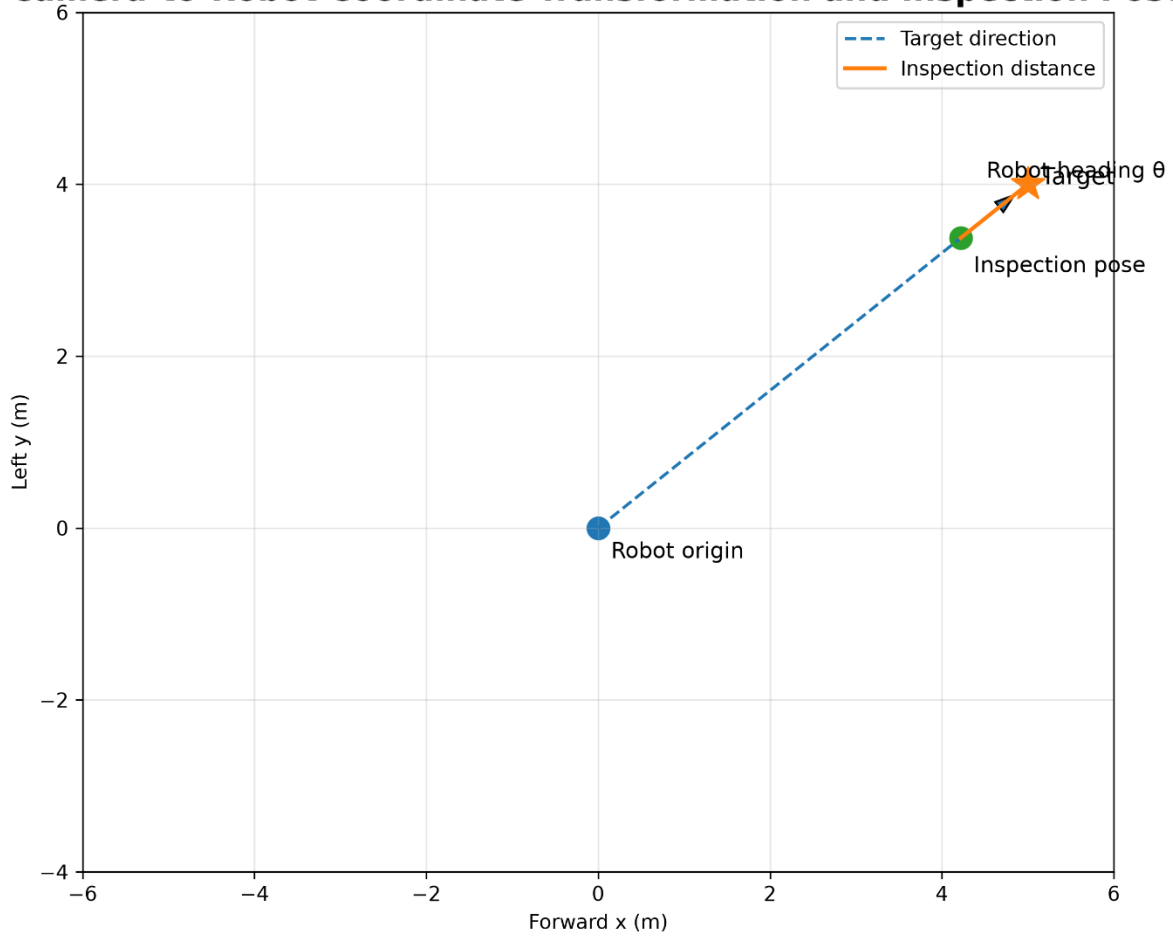


Figure 3. Camera and robot coordinate-frame transformation.

### Inspection Pose Generation

The robot does not navigate directly to the target center.

Instead, a suitable inspection pose is generated at a predefined preferred distance from the target.

Let the target position be:

$$P_t = [x_t \ y_t]$$

The distance to the target is:

$$d = \|p_t\|$$

The unit direction vector toward the target is:

$$\hat{d} = p_t / \|p_t\|$$

If  $d_i$  is the desired inspection distance, the inspection position is calculated as:

$$p_g = p_t - d_i \hat{d}$$

The desired heading is then calculated so that the robot faces the target:

$$\theta_g = \text{atan2}(y_t - y_g, x_t - x_g)$$

For the experimental scene, the resulting inspection goal is:

$$G = [4.219, 3.375, 0.675]$$

where:

- 4.219 m = forward position
- 3.375 m = left position
- 0.675 rad = desired heading

## Obstacle Representation

The identified table is represented as a rectangular obstacle in the robot coordinate system.

The obstacle is expanded by a safety margin to account for navigation uncertainty.

The resulting obstacle representation is:

$$x_{\min} = 1.85$$

$$x_{\max} = 3.65$$

$$y_{\min} = 0.85$$

$$y_{\max} = 3.15$$

Thus, the obstacle is represented as:

$$O = [x_{\min}, x_{\max}] \times [y_{\min}, y_{\max}]$$

This representation allows the NMPC controller to calculate the robot's distance from the obstacle.

## Obstacle Clearance Calculation

For a robot position  $(x, y)$ , the minimum Euclidean distance to a rectangular obstacle is calculated using:

$$dx = \max(x_{\min} - x, 0, x - x_{\max})$$

$$dy = \max(y_{\min} - y, 0, y - y_{\max})$$

The obstacle clearance is:

$$d_{\text{obs}} = \sqrt{dx^2 + dy^2}$$



A safety distance of:

$$d_{\text{safe}} = 0.5\text{m}$$

is maintained.

An additional influence distance of:

$$d_{\text{influence}} = 2.0\text{m}$$

causes the controller to begin considering the obstacle before the robot reaches the critical safety boundary.

### Safe Waypoint Planning

Since the direct path between the initial robot position and inspection pose intersects the table, direct navigation is unsafe.

Therefore, intermediate waypoints are generated.

For the final successful experiment, the system generated:

$$\text{WP1}=[1.10,0.10,0]$$

$$\text{WP2}=[4.40,0.10,0]$$

$$\text{WP3}=[4.219,3.375,0.675]$$

The navigation sequence is therefore:

Start

|



WP1

|

| Move below obstacle



WP2

|

| Move past obstacle



## WP3 / Inspection Pose

This approach allows the robot to move around the obstacle before approaching the target.

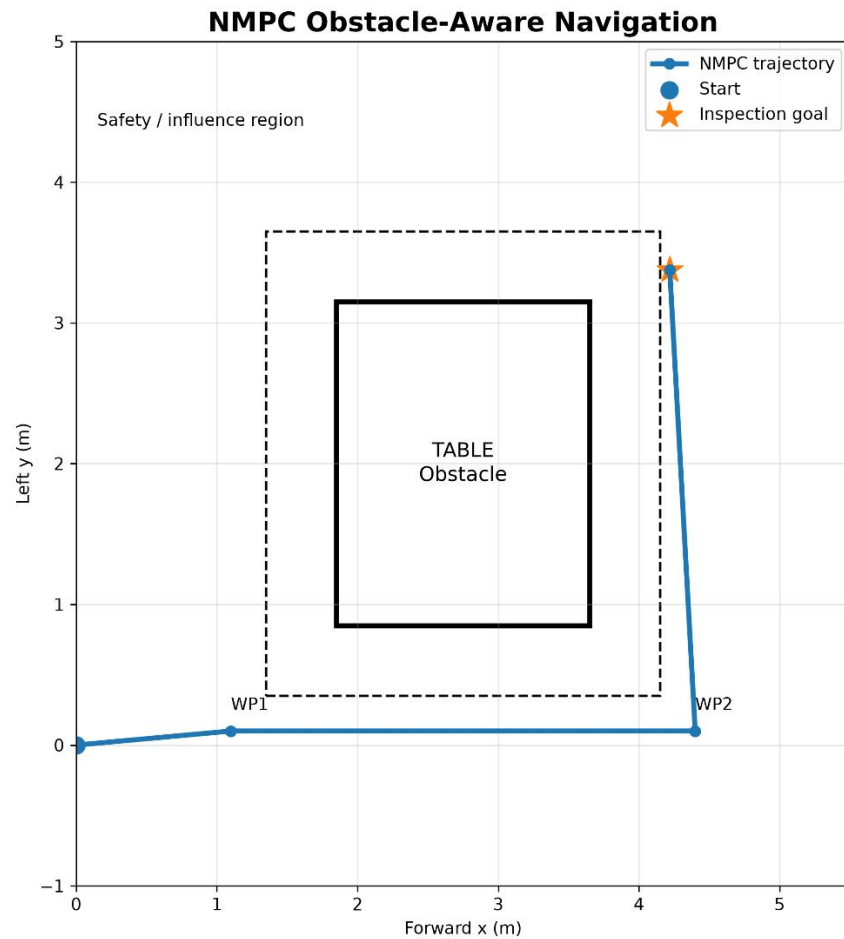


Figure 4. Planned collision-free navigation route around the table.

## Robot Motion Model

The robot is modeled using a nonlinear unicycle motion model.

The robot state is:

$$S = [x \ y \ \theta]$$

where:

- = forward position
- = lateral position
- = robot heading

The control input is:

$$\mathbf{u} = [\mathbf{v} \ \omega]$$

where:

- $\mathbf{v}$  = linear velocity
- $\omega$  = angular velocity

The continuous-time model is:

$$\dot{x} = v \cos(\theta)$$

$$\dot{y} = v \sin(\theta)$$

$$\dot{\theta} = \omega$$

Using a discrete timestep :

$$x_{k+1} = x_k + v_k \cos(\theta_k) \Delta t$$

$$y_{k+1} = y_k + v_k \sin(\theta_k) \Delta t$$

$$\theta_{k+1} = \theta_k + \omega_k \Delta t$$

The simulation uses:

$$\Delta t = 0.1 \text{ s}$$

with velocity limits:

$$|v| \leq 1.0 \text{ m/s}$$

$$|\omega| \leq 1.0 \text{ rad/s}$$

## Nonlinear Model Predictive Control

NMPC is responsible for generating the robot's control commands.

At every control step, the controller predicts the robot's future trajectory over a finite prediction horizon.

The final configuration uses:

$$N = 30$$

with :

$$\Delta t = 0.1 \text{ s}$$

Therefore, the prediction horizon is:

$$T = N \Delta t$$

$$T = 3.0s$$

The controller optimizes a sequence:

$$U = \{(v_0, \omega_0), \dots, (v_{N-1}, \omega_{N-1})\}$$

Only the first control action is applied to the robot, after which the state is updated and the optimization is performed again.

This is the receding-horizon principle.

### **NMPC Cost Function**

The optimization objective combines several terms.

The total cost can be represented as:

$$J = J_{\text{position}} + J_{\text{heading}} + J_{\text{obstacle}} + J_{\text{control}} + J_{\text{terminal}} + J_{\text{progress}}$$

#### **Position Cost**

The controller minimizes the distance between predicted position and goal:

$$J_p = w_p [(x-x_g)^2 + (y-y_g)^2]$$

#### **Heading Cost**

The heading error is:

$$e_\theta = \text{wrap}(\theta - \theta_g)$$

and the heading cost is:

$$J_\theta = w_\theta e_\theta^2$$

#### **Obstacle Influence Cost**

When:

$$d_{\text{obs}} < d_{\text{influence}}$$

the controller applies:

$$J_{\text{influence}} = w_i (d_{\text{influence}} - d_{\text{obs}})^2$$

This encourages the robot to move away from obstacles before entering the critical safety region.

## Safety Cost

If:

$$d_{obs} < d_{safe}$$

the stronger penalty becomes:

$$J_{safe} = w_o(d_{safe} - d_{obs})^2$$

This discourages unsafe trajectories.

## Control Cost

To avoid unnecessarily aggressive commands:

$$J_u = w_u(v^2 + \omega^2)$$

## Terminal Cost

At the end of the prediction horizon, additional penalties are applied:

$$J_T = w_T \|p_N - p_g\|^2 + w_T \theta e_{\theta}^2, N$$

The combined objective encourages the robot to approach the target accurately while remaining safe and using reasonable control effort.

## Receding-Horizon Optimization and Closed-Loop Control

At each control cycle, the NMPC controller solves a finite-horizon nonlinear optimization problem using the current robot state, the active navigation goal, and the obstacle map.

The optimization problem can be expressed as:

$$U^* = \arg \min_U J(U)$$

subject to the robot motion model and control constraints:

$$|v_k| \leq v_{max}$$

$$|\omega_k| \leq \omega_{max}$$

where represents the optimal sequence of linear and angular velocity commands.

For the proposed system:

$$v_{max} = 1.0 \text{m/s}$$

$$\omega_{max} = 1.0 \text{rad/s}$$

The nonlinear state prediction is performed for all prediction steps. The optimization is solved using the Sequential Least Squares Programming (SLSQP) method.

Only the first optimized control pair

$$u_k = [v_k, \omega_k]$$

is applied to the robot. The robot state is then updated according to the discrete motion model, and the optimization is repeated using the newly obtained state.

This repeated optimization provides feedback correction throughout navigation and allows the robot to react to changes in the predicted trajectory and obstacle clearance.

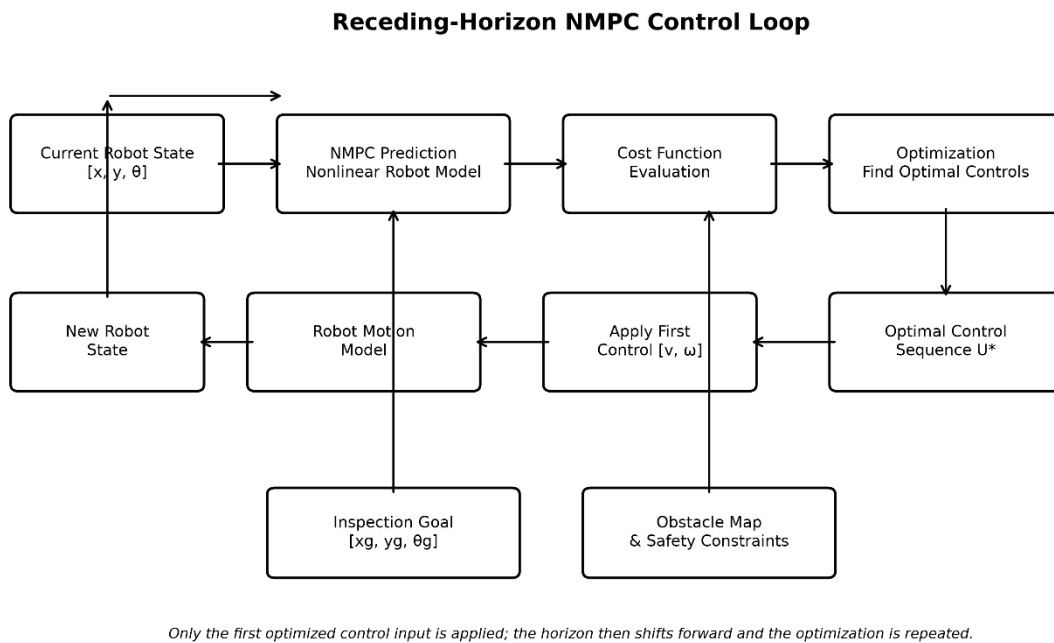


Figure 5. Receding-horizon NMPC control loop showing nonlinear prediction, cost-function evaluation, control optimization, first-control application, state update, and repeated optimization.

### Closed-Loop NMPC Execution

At each simulation timestep, the current robot state is supplied to the NMPC controller together with the active navigation goal and obstacle map. The controller predicts the robot's future states over the specified prediction horizon and determines the control sequence that minimizes the combined objective function.

Although an entire sequence of control inputs is optimized, only the first control input is applied to the robot. The robot state is then updated using the nonlinear unicycle motion model. The updated state is fed back into the controller, and the optimization is repeated.

This receding-horizon strategy allows the controller to continuously react to the current robot state and environmental constraints rather than following a fixed open-loop trajectory.

The closed-loop operation can be summarized as:

$$S_k \rightarrow \text{NMPC optimization} \rightarrow U_k^* \rightarrow (v_k, \omega_k) \rightarrow S_{k+1} \rightarrow \text{NMPC optimization}$$

where  $S_k$  represents the current robot state and  $U_k^*$  represents the optimized control sequence.

### **Navigation and Waypoint Execution**

The navigation system operates in two levels. The high-level planning layer determines a sequence of safe navigation waypoints, while the NMPC controller provides continuous low-level motion control between successive waypoints.

When the direct path from the initial position to the inspection pose is blocked by an obstacle, intermediate waypoints are generated outside the expanded obstacle region. The current waypoint becomes the active NMPC goal.

Once the robot reaches the waypoint within a predefined positional tolerance, the controller switches to the next waypoint. This process continues until the final inspection pose is reached.

For the experimental scene, the successful navigation sequence was:

$$S_0 \rightarrow \text{WP1} \rightarrow \text{WP2} \rightarrow \text{WP3}$$

Where:

$$\text{WP1} = [1.10, 0.10, 0]$$

$$\text{WP2} = [4.40, 0.10, 0]$$

$$\text{WP3} = [4.219, 3.375, 0.675]$$

The first waypoint moves the robot below the obstacle, the second waypoint moves it beyond the obstacle, and the final waypoint corresponds to the desired inspection pose.

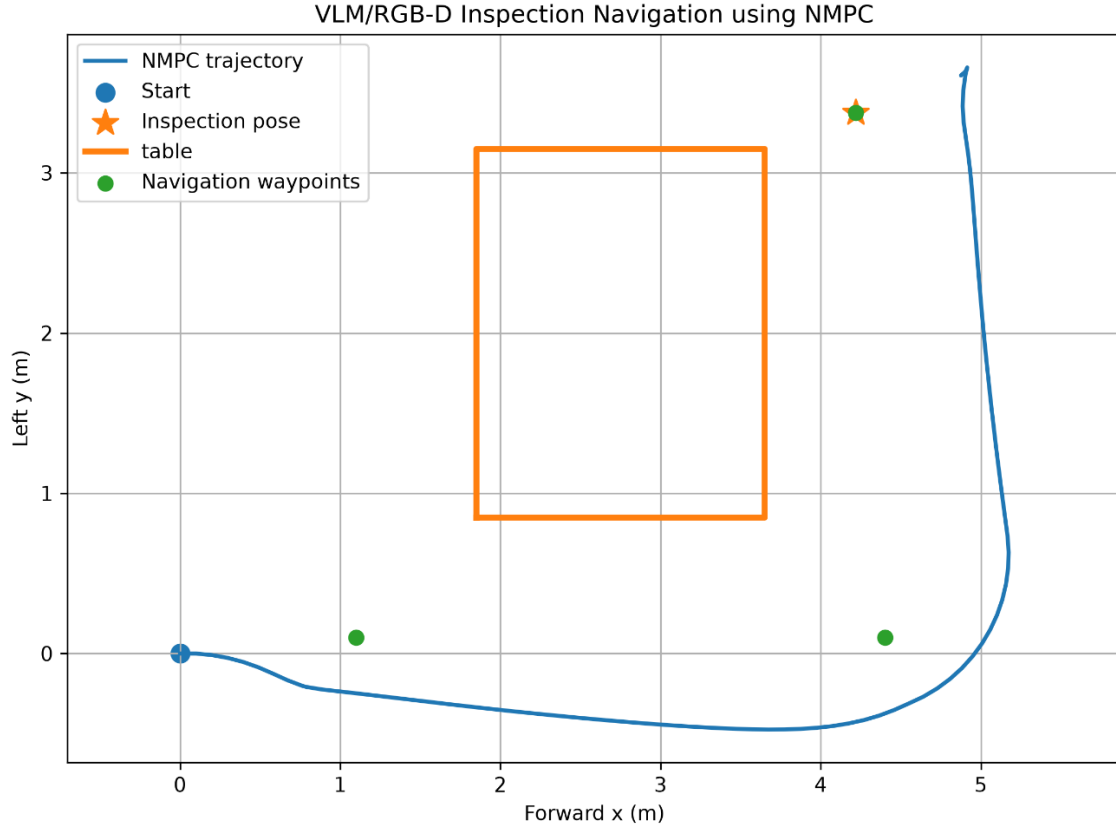


Figure 6. NMPC-generated robot trajectory showing collision-free navigation around the table obstacle toward the target inspection pose.

### Goal Switching and Termination

During navigation, the controller continuously evaluates the distance between the robot and the active waypoint. When the robot enters the specified waypoint tolerance region, the navigation system marks the waypoint as reached and activates the next goal.

For the final inspection pose, navigation is considered successful when both the position and heading errors satisfy their respective tolerances:

$$e_p = \sqrt{(x-x_g)^2 + (y-y_g)^2}$$

and

$$e_\theta = |\text{wrap}(\theta - \theta_g)|$$

The inspection goal is considered reached when:

$$e_p < \epsilon_p$$

and

$$e_\theta < \epsilon_\theta$$



where  $\epsilon_p$  is the position tolerance and  $\epsilon_\theta$  is the heading tolerance.

In the implemented system, the position tolerance is m and the heading tolerance is 8 deg.

### Safety and Collision Monitoring

Obstacle safety is monitored throughout the simulation using the minimum Euclidean clearance between the robot and each obstacle.

At every control step, the distance between the robot position and the expanded obstacle rectangles is calculated. The minimum value across all obstacles is recorded as the instantaneous obstacle clearance.

The minimum clearance observed during the complete trajectory is used as an important safety metric:

$$d_{\min} = \min d_{\text{obs},k}$$

A collision is recorded whenever:

$$d_{\text{obs}} \leq 0$$

The system therefore evaluates not only whether the robot reaches the target, but also whether it maintains safe separation from obstacles throughout the navigation process.

### Simulation and Experimental Configuration

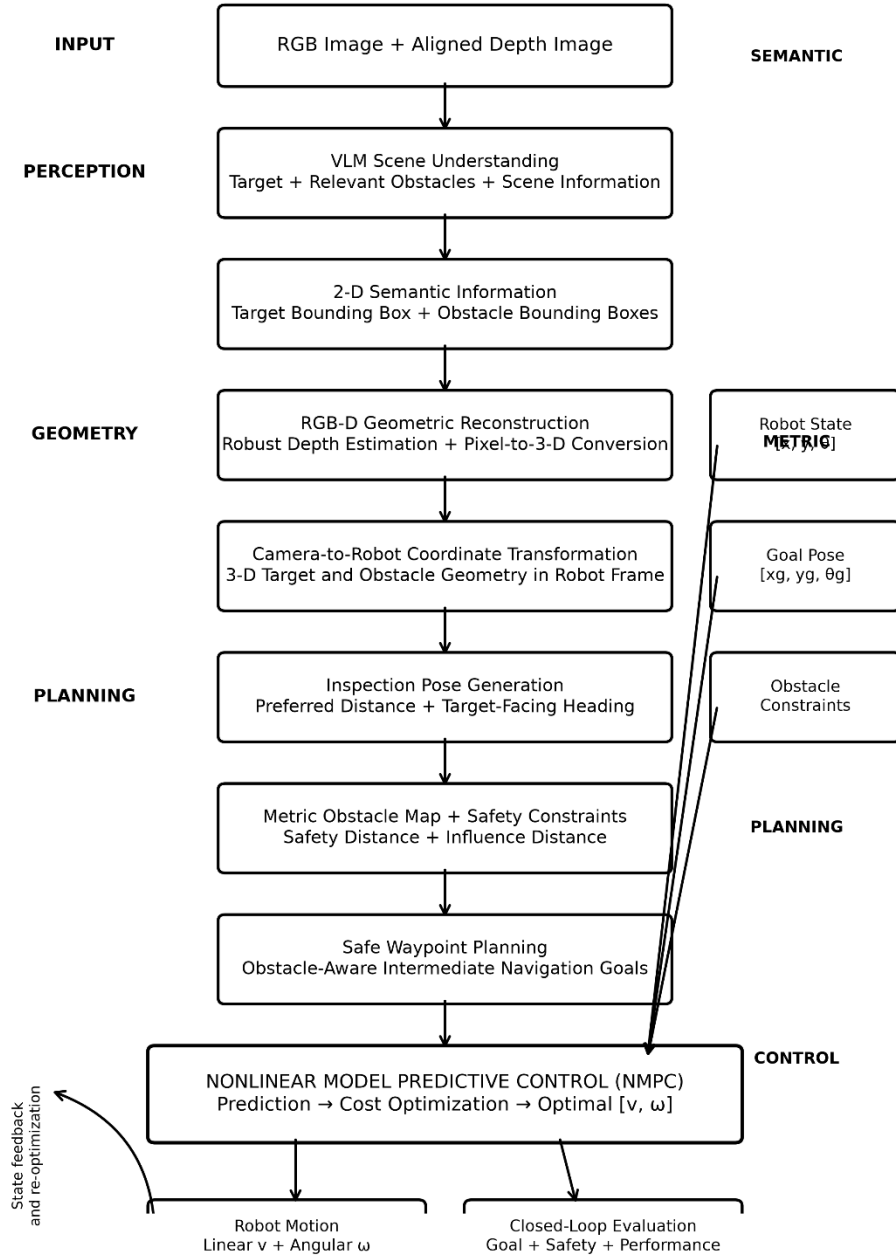
The complete perception-to-control pipeline is evaluated using the synthetic RGB-D inspection scene. The scene contains a chair as the inspection target and a table acting as the navigation obstacle.

The principal experimental parameters are summarized in Table 1.

| Parameter                   | Value     |
|-----------------------------|-----------|
| Robot model                 | Unicycle  |
| NMPC prediction horizon     | 30 steps  |
| Sampling time               | 0.1 s     |
| Prediction horizon          | 3.0 s     |
| Maximum linear velocity     | 1.0 m/s   |
| Maximum angular velocity    | 1.0 rad/s |
| Safety distance             | 0.5 m     |
| Obstacle influence distance | 2.0 m     |
| Position tolerance          | 0.12 m    |
| Heading tolerance           | 8°        |
| Target                      | Chair     |
| Main obstacle               | Table     |

## Complete VLM-Based Mobile Robot Inspection Pipeline

Perception → Geometry → Planning → NMPC Control → Evaluation



The system forms a closed perception-to-action loop in which visual information is converted into metric navigation goals and continuously optimized robot motion.

Figure 7. Complete end-to-end perception-to-action pipeline for VLM-based mobile robot inspection.

## Results and Performance Evaluation

The proposed VLM-based nonlinear navigation and inspection system was evaluated using the synthetic RGB-D inspection scene containing a chair as the target object and a table as the

navigation obstacle. The complete pipeline was executed from scene perception through 3-D reconstruction, inspection-pose generation, waypoint planning, NMPC control, and final inspection.

The final successful execution demonstrated collision-free navigation with both the planned waypoints and the inspection goal reported as reached.

### Scene Perception Results

The perception module identified the chair as the target object and the table as the relevant obstacle.

The detected target information was:

| Parameter                   | Value               |
|-----------------------------|---------------------|
| Target                      | Chair               |
| Bounding box                | [85, 145, 155, 270] |
| Robot-frame target position | [5.0, 4.0] m        |

The relevant obstacle was identified as a table with the image-space bounding box:

[70, 150, 236, 376]

The perception module also identified that the direct path between the initial robot position and the target inspection region was blocked.

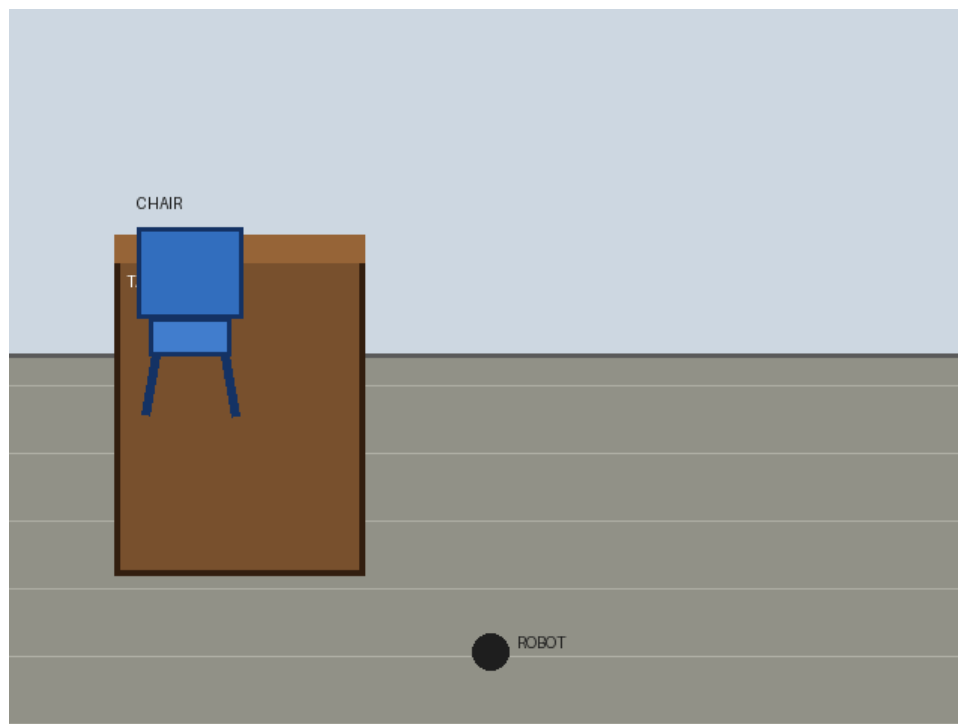


Figure 8. Experimental RGB scene containing the detected inspection target and navigation obstacle.

### 3-D Target Localization

The RGB-D processing module converted the detected target bounding box into a metric 3-D position using the depth information and camera intrinsic parameters.

The estimated target center in the camera coordinate system was:

$$P \text{ camera target} = [-4.00, -0.66, 5.00] \text{ m}$$

Using the camera-to-robot transformation,

$$x = Z, \quad y = -X$$

the corresponding target position in the robot coordinate system was:

$$P \text{ robot target} = [5.00, 4.00] \text{ m}$$

This indicates that the chair was located approximately 5 m in front of the robot and 4 m to its left.

### Inspection Pose Generation

Rather than navigating directly to the target center, the system generated an inspection pose at a safe observation distance from the chair.

The calculated inspection pose was:

$$G = [4.219, 3.375, 0.675]$$

where:

- $x_g = 4.219 \text{ m}$  is the forward position,
- $y_g = 3.375 \text{ m}$  is the lateral position,
- $\theta_g = 0.675 \text{ rad}$  is the desired heading.

The resulting pose places the robot near the target while orienting the robot toward the chair for inspection.

### Obstacle-Aware Navigation

The table was converted into an expanded rectangular obstacle in the robot coordinate system:

$$x \text{ min} = 1.85 \quad x \text{ max} = 3.65$$

$$y \text{ min} = 0.85 \quad y \text{ max} = 3.15$$

Since the direct path was blocked, the navigation system generated three safe intermediate goals:

$$WP1 = [1.10, 0.10, 0]$$

WP2=[4.40,0.10,0]

WP3=[4.219,3.375,0.675]

The robot therefore followed the route:

Start→WP1→WP2→WP3

The execution log confirmed:

[Navigation] Reached WP1.

[Navigation] Reached WP2.

and the final result reported:

inspection\_goal\_reached: True

waypoint\_reached: True

## Navigation Performance

The main navigation performance results obtained from the final successful simulation are presented in Table 2.

**Table 2. Navigation Performance Metrics**

| <b>Metric</b>              | <b>Result</b> |
|----------------------------|---------------|
| Inspection goal reached    | <b>True</b>   |
| Waypoint reached           | <b>True</b>   |
| Navigation time            | 80.0 s        |
| Control steps              | 800           |
| Path length                | 8.933 m       |
| Final position error       | 0.696 m       |
| Final heading error        | 0.061°        |
| Minimum obstacle clearance | 1.179 m       |
| Collision count            | <b>0</b>      |

The results demonstrate that the robot successfully completed the planned navigation sequence without collision. The minimum observed obstacle clearance was approximately 1.18 m, which is substantially greater than the predefined safety distance of 0.5 m.

The collision count was zero throughout the complete simulation, demonstrating that the obstacle-aware planning and NMPC safety formulation successfully prevented contact with the table.

## Velocity and Control Behaviour

The NMPC controller generated bounded linear and angular velocity commands throughout the navigation process.

The measured control statistics are shown below.

**Table 3. NMPC Control Statistics**

| Parameter                         | Result       |
|-----------------------------------|--------------|
| Average absolute linear velocity  | 0.112 m/s    |
| Maximum absolute linear velocity  | 1.000 m/s    |
| Average absolute angular velocity | 0.0467 rad/s |
| Maximum absolute angular velocity | 1.000 rad/s  |

The maximum linear and angular velocities remained within the specified controller limits:

$$|v| \leq 1.0 \text{ m/s}$$

$$|\omega| \leq 1.0 \text{ rad/s}$$

This confirms that the control inputs remained within the predefined actuator constraints.

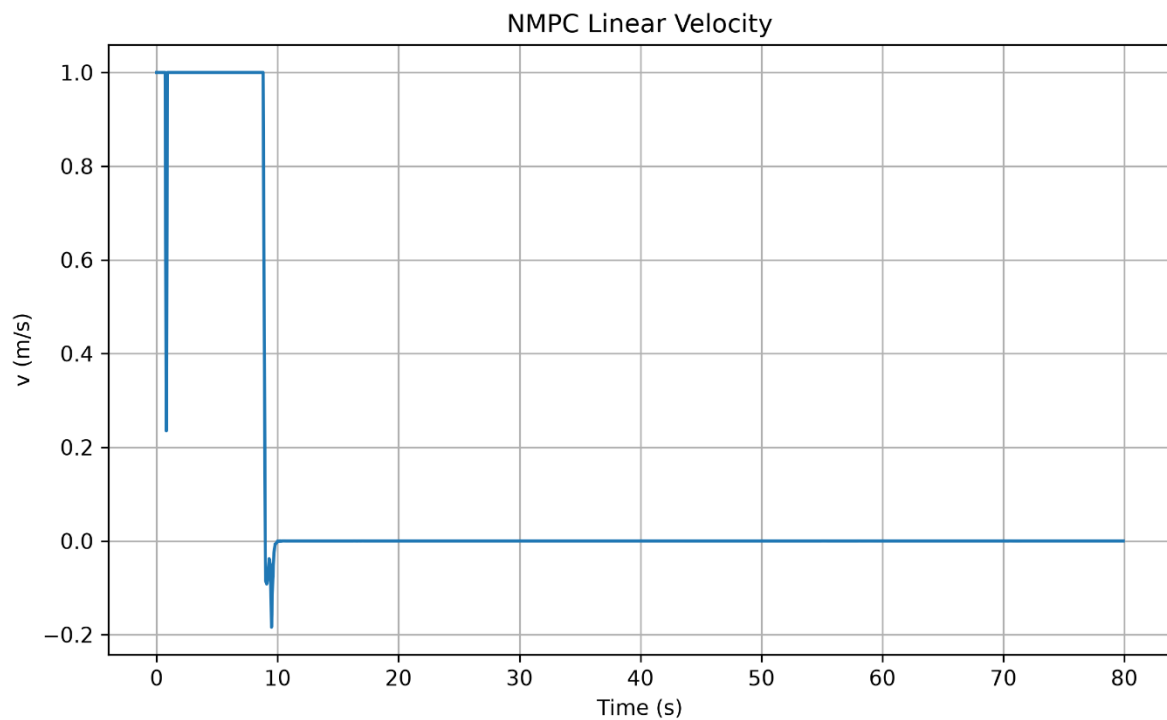


Figure 9. Linear velocity generated by the NMPC controller during navigation.

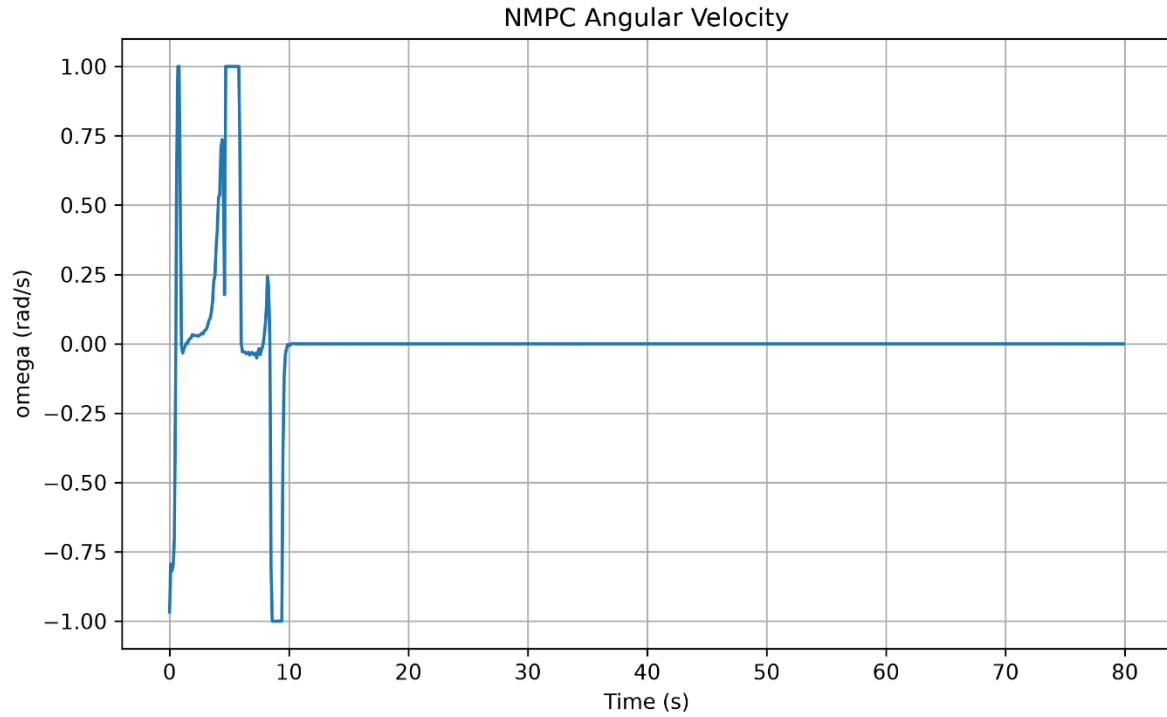


Figure 10. Angular velocity generated by the NMPC controller during navigation.

### Obstacle Clearance Analysis

Obstacle clearance was continuously monitored during navigation.

The minimum obstacle clearance obtained was:

$$d_{\min} = 1.1786 \text{ m}$$

while the predefined safety distance was:

$$d_{\text{safe}} = 0.5 \text{ m}$$

Therefore,

$$d_{\min} > d_{\text{safe}}$$

which indicates that the robot maintained a safe separation from the obstacle during the complete trajectory.

The clearance profile provides additional evidence that the robot did not enter the collision region.

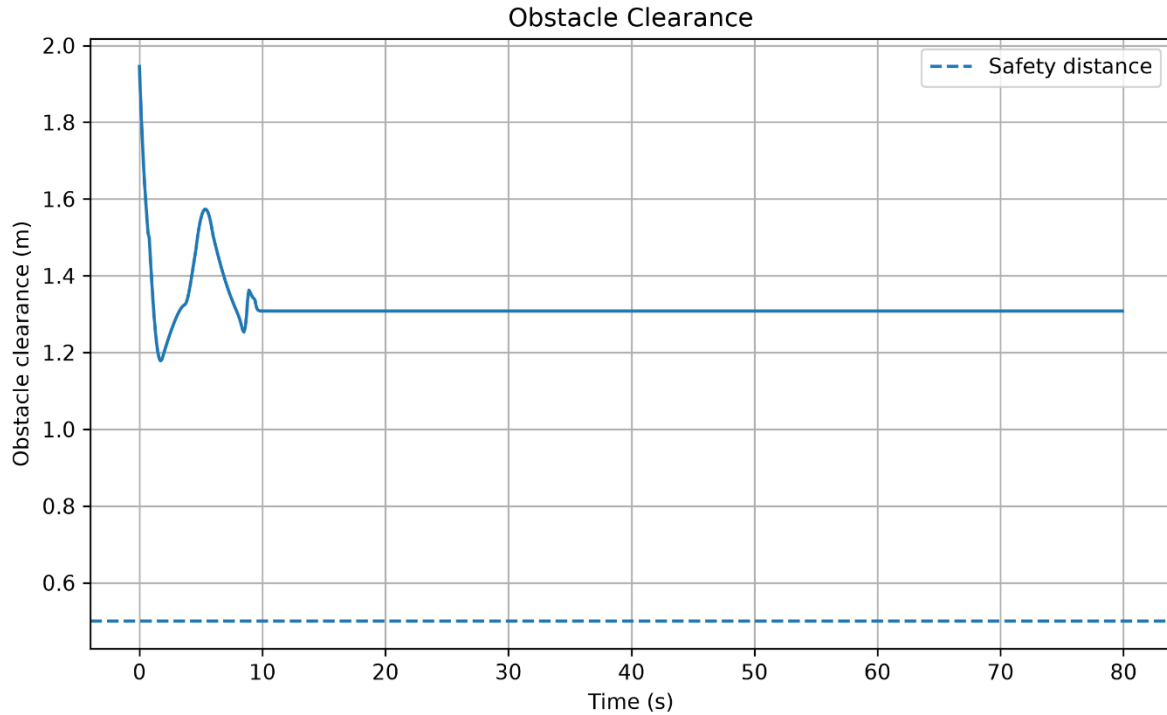


Figure 11. Obstacle-clearance profile during NMPC navigation with the predefined safety-distance threshold.

### NMPC Computational Performance

The computational performance of the controller was also evaluated.

The measured NMPC computation statistics were:

| Metric                        | Result    |
|-------------------------------|-----------|
| Average NMPC computation time | 306.38 ms |
| Maximum NMPC computation time | 983.25 ms |

The average computation time represents the mean time required to solve the NMPC optimization at each control step during the simulation.

The computation time demonstrates the computational cost associated with nonlinear prediction, obstacle-aware cost evaluation, and optimization over the prediction horizon.

For the present simulation-based evaluation, the controller successfully completed the complete navigation task despite the computational cost of nonlinear optimization.

### Overall System Performance

The final experiment demonstrates the complete operation of the proposed system:



The robot successfully identified the target, constructed the corresponding metric representation, planned a safe route around the obstacle, reached the required navigation waypoints, and satisfied the final inspection-goal condition.

The most important outcome is that the system achieved:

- **Successful inspection goal:** Yes
- **Successful waypoint navigation:** Yes
- **Collision:** 0
- **Minimum clearance:** 1.179 m
- **Maximum linear velocity:** 1.0 m/s
- **Maximum angular velocity:** 1.0 rad/s

Overall, the experimental results demonstrate the feasibility of integrating semantic visual perception, RGB-D geometric reasoning, obstacle-aware planning, and NMPC into a unified mobile robot inspection framework.

## **Discussion of the Proposed Framework**

The experimental results demonstrate that the proposed perception-to-action framework can integrate semantic scene understanding, RGB-D geometric estimation, obstacle-aware planning, and nonlinear model predictive control into a unified robotic inspection pipeline.

The system successfully transformed visual information into metric spatial information and subsequently used this information to generate a collision-free navigation strategy. The detected chair was converted from image coordinates into a 3-D camera-frame position and then transformed into the robot coordinate frame. This enabled the system to reason about the target in physical metric space rather than relying only on image-space information.

A significant feature of the proposed framework is the generation of an inspection pose rather than directly navigating to the target object. The robot approaches the target from a specified distance and calculates its final orientation such that the target remains in front of the robot. This is more appropriate for inspection-oriented tasks than simply minimizing the distance to the detected object.

The obstacle-aware waypoint mechanism further improves navigation in environments where a direct target path is blocked. In the experimental scene, the table obstructs the direct route to the chair. The planner therefore generates intermediate waypoints that guide the robot around the obstacle before approaching the final inspection position.

The NMPC controller provides closed-loop motion control by repeatedly predicting future robot states and optimizing the control inputs. Obstacle clearance is included directly in the optimization objective, allowing the controller to account for safety while simultaneously minimizing navigation error and control effort.

## **Safety Performance**

The final experimental run achieved a collision count of zero and maintained a minimum obstacle clearance of approximately 1.18 m. This is greater than the configured safety distance

of 0.5 m, indicating that the generated navigation trajectory remained safely separated from the obstacle throughout the simulation.

The obstacle influence region of 2.0 m also allows the controller to begin responding to obstacles before reaching the critical safety boundary. This provides a smoother alternative to reacting only when the robot is already close to an obstacle.

## Computational Performance

The NMPC controller operated in a receding-horizon manner throughout the simulation. The reported average optimization time was approximately 306 ms, while the maximum recorded computation time was approximately 983 ms.

These results demonstrate that the proposed controller can perform nonlinear trajectory optimization for the simulated inspection task. However, the computation time indicates that further optimization would be required for deployment on a computationally constrained real-time robotic platform.

## Limitations

The current implementation has several limitations.

1. **Synthetic perception evaluation:** The final benchmark uses the synthetic ground-truth perception interface rather than a fully autonomous VLM-generated scene description.
2. **Single-scene evaluation:** The current experiment evaluates the system on a limited synthetic environment containing one target and one primary obstacle.
3. **Simplified robot model:** The robot is represented using a unicycle motion model and therefore does not capture the complete dynamics, wheel constraints, actuator limitations, or mechanical characteristics of a physical mobile robot.
4. **Static environment:** The obstacle is stationary during the experiment. Dynamic obstacles and moving targets are not currently considered.
5. **Simulation-to-reality gap:** Real-world deployment would introduce additional sources of error, including RGB-D noise, camera calibration errors, localization uncertainty, actuator disturbances, and imperfect perception.
6. **Computational cost:** NMPC optimization requires significant computation, particularly as the prediction horizon and number of obstacles increase.

## Future Work

### Real VLM Integration

Future work will replace the synthetic perception interface with a fully operational VLM-based perception module capable of identifying targets and relevant obstacles directly from real RGB images using natural-language inspection instructions.

### Real-Time RGB-D Navigation

The system can be extended to process continuously acquired RGB-D frames rather than using a fixed scene. This would allow the robot to update the obstacle map and inspection target location during navigation.

### **Dynamic Obstacle Avoidance**

Future versions can incorporate moving obstacles and dynamic obstacle prediction into the NMPC formulation. This would allow the robot to safely operate in more realistic environments.

### **Improved NMPC Optimization**

The computational performance can be improved using faster nonlinear optimization solvers, warm-starting, reduced prediction horizons, model simplification, or hardware acceleration.

### **Physical Robot Deployment**

The final stage would involve transferring the proposed framework from simulation to a physical mobile robot. Real-world experiments could evaluate the effects of sensor noise, localization uncertainty, actuator constraints, and environmental disturbances.

### **Multi-Object Inspection**

The framework can also be extended to multiple targets, allowing the robot to identify, prioritize, and sequentially inspect several objects within the same environment.

## **Applications**

The proposed VLM-based nonlinear navigation and inspection framework can be applied to several robotic inspection and autonomous navigation scenarios.

### **Industrial Inspection**

The system can be used for autonomous inspection of machines, equipment, pipelines, electrical panels, and other industrial assets. A robot can identify an object of interest from visual information and navigate to an appropriate inspection position while avoiding surrounding equipment.

### **Infrastructure Inspection**

The framework can support inspection of bridges, buildings, tunnels, and other infrastructure. RGB-D perception can provide spatial information about structural components, while NMPC can generate safe navigation trajectories around obstacles.

### **Warehouse and Indoor Robotics**

The proposed approach can be applied to indoor mobile robots operating in warehouses, laboratories, offices, and manufacturing environments. Semantic understanding allows the robot to identify objects of interest while geometric reasoning enables obstacle-aware navigation.

### **Search and Inspection Operations**

The system can be used when a robot needs to locate and approach a particular object specified using natural language, such as *"inspect the chair"* or *"inspect the equipment near the table."* This provides a more flexible interaction mechanism than systems requiring manually specified coordinates.

### **Service and Assistive Robotics**

The framework can potentially support service robots that need to approach specific objects or locations while maintaining safe distances from obstacles. The inspection-pose generation mechanism can be adapted for tasks such as object observation, monitoring, and visual verification.

### **Autonomous Robotic Inspection**

The overall perception-to-action architecture can serve as a foundation for autonomous inspection systems in which visual understanding, spatial reasoning, path planning, and motion control are integrated into a single pipeline.

## **Expected Deliverables**

The major deliverables of the project are as follows.

### **Integrated Perception-to-Action Pipeline**

A complete software pipeline integrating:

RGB/RGB-D Input → Scene Perception → 3-D Reconstruction → Coordinate Transformation  
→ Inspection Pose Generation → Waypoint Planning → NMPC Control → Evaluation

### **Scene Perception Module**

A perception module capable of identifying the target object, relevant obstacles, bounding boxes, and navigation-related scene information from the input scene.

### **RGB-D Geometric Processing Module**

A module for extracting robust depth information and converting image-space observations into metric 3-D coordinates using camera calibration parameters.

### **Inspection Pose Generation**

An automatic inspection-pose generation mechanism that determines:

- Safe distance from the target
- Target-relative inspection position
- Required robot heading

### **Obstacle-Aware Navigation Module**

A metric obstacle representation and waypoint-generation mechanism capable of producing safe intermediate navigation points when the direct route to the target is blocked.

### **NMPC Controller**

A nonlinear model predictive controller that generates robot linear and angular velocity commands while considering:

- Position error
- Heading error
- Obstacle clearance
- Safety distance
- Control effort
- Terminal accuracy
- Navigation progress

### **Simulation and Evaluation Results**

A complete simulation demonstrating target inspection and obstacle avoidance, together with quantitative evaluation using:

- Inspection goal achievement
- Position error
- Heading error
- Path length
- Minimum obstacle clearance
- Collision count
- Velocity statistics
- NMPC computation time

### **Generated Visualizations**

The project produces visualization outputs including:

- Robot navigation trajectory
- Obstacle representation
- Inspection pose
- Linear velocity profile
- Angular velocity profile
- Obstacle-clearance profile

### **Quantitative Result Files**

The system generates machine-readable evaluation files including:

- scene\_result.json

- metrics.csv
- trajectory.csv
- controls.csv

These files provide reproducible records of the navigation experiment.

## Technical Documentation

The final deliverable also includes detailed documentation describing the system architecture, mathematical formulation, implementation methodology, experimental setup, results, limitations, and future development.

## References

- [1] Tian Wang , Junming Fan , [Pai Zheng](#)<sup>a</sup>, [Ruqiang Yan](#)<sup>b</sup>, [Lihui Wang](#)<sup>c</sup> “Vision-Language Model-Based Human-Guided Mobile Robot Navigation in an Unstructured Environment for Human-Centric Smart Manufacturing,” <https://doi.org/10.1016/j.eng.2025.04.028>.
- [2] H. Lim, Y. Kang, C. Kim, J. Kim, and B.-J. You, “Nonlinear model predictive controller design with obstacle avoidance for a mobile robot,” in *Proc. IEEE/ASME Int. Conf. Mechatronic and Embedded Systems and Applications (MESA)*, 2008, pp. 494–499, doi: 10.1109/MESA.2008.4735699.
- [3] Yingbai Hu ,Hang Su ,Longbin Zhang ,Shu Miao ,Guang Chen , and Alois Knoll , “Nonlinear Model Predictive Control for Mobile Robot Using Varying-Parameter Convergent Differential Neural Network,” <https://doi.org/10.3390/robotics8030064>.
- [4] W. Chen, S. Hu, R. Talak, and L. Carlone, “Leveraging large (visual) language models for robot 3D scene understanding,” *arXiv preprint arXiv:2209.05629*, 2022.
- [5] X. Liu and V. Atanassov, “Safe model predictive control approach for non-holonomic mobile robots,” *arXiv preprint arXiv:2207.12878*, 2022.
- [6] A. Sathya, P. Sopasakis, R. Van Parys, A. Themelis, G. Pipeleers, and P. Patrinos, “Embedded nonlinear model predictive control for obstacle avoidance using PANOC,” *arXiv preprint arXiv:1904.10546*, 2019.
- [7] O. Y. Ismael, M. Almaged, and A. I. Abdulla, “Nonlinear model predictive control-based collision avoidance for mobile robot,” *J. Robot. Control*, vol. 5, no. 1, pp. 142–151, Jan. 2024, doi: 10.18196/jrc.v5i1.20615.
- [8] A. Radford *et al.*, “Learning transferable visual models from natural language supervision,” in *Proc. 38th Int. Conf. Machine Learning (ICML)*, 2021, pp. 8748–8763.
- [9] J. Li *et al.*, “BLIP-2: Bootstrapping language-image pre-training with frozen image encoders and large language models,” in *Proc. 40th Int. Conf. Machine Learning (ICML)*, 2023, pp. 19730–19742.

- [10] Z. Shao *et al.*, “DeepSeek-VL: Towards real-world vision-language understanding,” *arXiv preprint arXiv:2403.05525*, 2024.
- [11] M. A. Fischler and R. C. Bolles, “Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography,” *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, Jun. 1981.
- [12] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. Pearson, 2018.
- [13] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. Cambridge, MA, USA: MIT Press, 2005.
- [14] D. Fox, W. Burgard, and S. Thrun, “The dynamic window approach to collision avoidance,” *IEEE Robotics & Automation Magazine*, vol. 4, no. 1, pp. 23–33, Mar. 1997.
- [15] J. Nocedal and S. J. Wright, *Numerical Optimization*, 2nd ed. New York, NY, USA: Springer, 2006.
- [16] J. B. Rawlings, D. Q. Mayne, and M. M. Diehl, *Model Predictive Control: Theory, Computation, and Design*, 2nd ed. Madison, WI, USA: Nob Hill Publishing, 2017.
- [17] E. F. Camacho and C. Bordons, *Model Predictive Control*, 2nd ed. London, U.K.: Springer, 2007.
- [18] G. Williams *et al.*, “Information theoretic MPC for model-based reinforcement learning,” in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2017, pp. 1714–1721.
- [19] F. F. Lewis, D. M. Dawson, and C. T. Abdallah, *Robot Manipulator Control: Theory and Practice*, 2nd ed. New York, NY, USA: Marcel Dekker, 2004.
- [20] B. Siciliano, L. Sciavicco, L. Villani, and G. Oriolo, *Robotics: Modelling, Planning and Control*. London, U.K.: Springer, 2009.
- [21] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, “Robot Operating System 2: Design, architecture, and uses in the wild,” *Science Robotics*, vol. 7, no. 66, 2022.
- [22] N. D. Ratliff, M. Zucker, J. A. Bagnell, and S. Srinivasa, “CHOMP: Gradient optimization techniques for efficient motion planning,” in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2009, pp. 489–494.